

A PROJECT REPORT ON

AI POWERED DEBT RELIEF & FINANCIAL RECOVERY PLATFORM

(FinRelief AI)

A report submitted in partial fulfilment of the requirements of the
APSCHE Internship Programme

Submitted by

1. **Gude Lohitha - 23P31A42G1**
2. **Mani Prabha Bhuvanasi - 23P31A42F2**

Aditya college of engineering and technology
Andhra Pradesh State Council of Higher Education (APSCHE)
2025 – 2026

TABLE OF CONTENTS

1. Introduction	1
1.1 Project Overview	1
1.2 Purpose	2
2. Ideation Phase.....	3
2.1 Problem Statement.....	3
2.2 Empathy Map Canvas.....	4
2.3 Brainstorming	5
3. Requirement Analysis.....	7
3.1 Customer Journey Map.....	7
3.2 Solution Requirement	8
3.3 Data Flow Diagram	10
3.4 Technology Stack.....	11
4. Project Design	13
4.1 Problem Solution Fit.....	13
4.2 Proposed Solution.....	14
4.3 Solution Architecture.....	15
5. Project Planning & Scheduling.....	17
5.1 Project Planning.....	17
6. Functional and Performance Testing	19
6.1 Performance Testing	19
7. Results	21
7.1 Output Screenshots	21
8. Advantages & Disadvantages	25
9. Conclusion.....	26
10. Future Scope.....	27
11. Appendix	28

1. INTRODUCTION

1.1 Project Overview

FinRelief AI is an AI-assisted web application designed to help individual borrowers understand, organise, and manage their personal debt in one place. Many people today juggle multiple loans — personal loans, credit cards, vehicle loans — across different lenders, and rarely have a single, simple view of how much they owe, how overdue they are, and how those loans affect their monthly finances. FinRelief AI addresses this gap by combining a loan management system with a rule-based financial analysis engine and an AI-assisted negotiation letter generator.

The platform is built as a three-tier web application. The frontend, developed using React.js and Vite, provides the user interface through which a borrower registers, logs in, and manages their loan and financial information. The backend, built using FastAPI (Python), exposes a REST API that performs all the business logic — user authentication, financial ratio calculations, loan record management, settlement prediction, and negotiation letter generation. Data is persisted using SQLite through the SQLAlchemy ORM, and the Google Gemini API is included in the technology stack to support AI-assisted natural language generation for the negotiation letters.

At its core, the system takes basic financial inputs from the user — monthly income, monthly expenses, and details of each loan (lender, loan type, outstanding amount, interest rate, EMI, and overdue months) — and converts them into meaningful, actionable insight: a debt-to-income (DTI) ratio, an EMI ratio, a financial stress classification, a suggested settlement percentage, and a ready-to-send settlement negotiation letter addressed to the lender.

1.2 Purpose

The purpose of building FinRelief AI is to give an ordinary borrower — someone without financial or legal expertise — the tools to understand their debt situation and take the first step toward resolving it, without needing to hire a financial consultant. Specifically, the project aims to:

- Provide a single, centralised place where a user can record and track every loan they hold, instead of relying on scattered paper statements or memory.
- Automatically calculate key financial health indicators — EMI ratio, debt-to-income (DTI) ratio, and monthly surplus — so that users can objectively see how stressed their finances are.
- Classify a user's financial stress level (Low / Medium / High) using clear, rule-based thresholds, so that the feedback is transparent and explainable rather than a hidden “black box” score.
- Estimate a realistic loan settlement amount by factoring in how overdue a loan is, its interest rate, and the borrower's disposable income, giving the user a data-informed starting point before they approach a lender.
- Automatically draft a formally worded settlement negotiation letter using the calculated settlement figures, saving the user the time and difficulty of writing one from scratch.

2. IDEATION PHASE

2.1 Problem Statement

The problem addressed by this project can be framed using a standard problem-statement template, connecting the affected user, their need, and the reason behind that need:

Element	Description
I am (User)	A salaried or self-employed individual who has taken one or more loans (personal loan, credit card, vehicle loan, etc.) from different lenders.
I want to	Understand my total outstanding debt, know how financially stressed I am, and find a realistic way to settle overdue loans with my lenders.
Because	I do not have a single place to track all my loans, I do not know how to calculate my debt-to-income ratio, and I do not know how to approach a lender to negotiate a settlement or write a formal request letter.

In simple terms: borrowers with multiple, overdue loans lack an easy way to (a) see their complete debt picture, (b) assess how serious their situation is, and (c) begin a structured negotiation with their lender. FinRelief AI is designed to solve exactly this problem using a web-based, self-service platform.

2.2 Empathy Map Canvas

An empathy map was used during the ideation phase to better understand the target user — a borrower struggling to manage multiple overdue loans.

Quadrant	Observations
SAYS	“I don't know how much I actually owe across all my loans.” “I'm scared to call the bank.” “I don't know if I can afford a settlement.”
THINKS	“Am I going to default?” “Will my credit score be ruined?” “I wish someone could just tell me a fair settlement number.”
DOES	Keeps loan statements in different apps/paper, avoids lender calls, delays payments further, searches the internet for generic advice.
FEELS	Anxious, overwhelmed, embarrassed to ask for help, unsure of financial jargon like DTI or EMI ratio.
PAIN	No consolidated view of debts; no idea how to calculate affordability; no confidence or template to negotiate with a lender.
GAIN	A simple dashboard of all loans, an easy-to-understand stress indicator, a fair settlement estimate, and a ready-made letter to send to the lender.

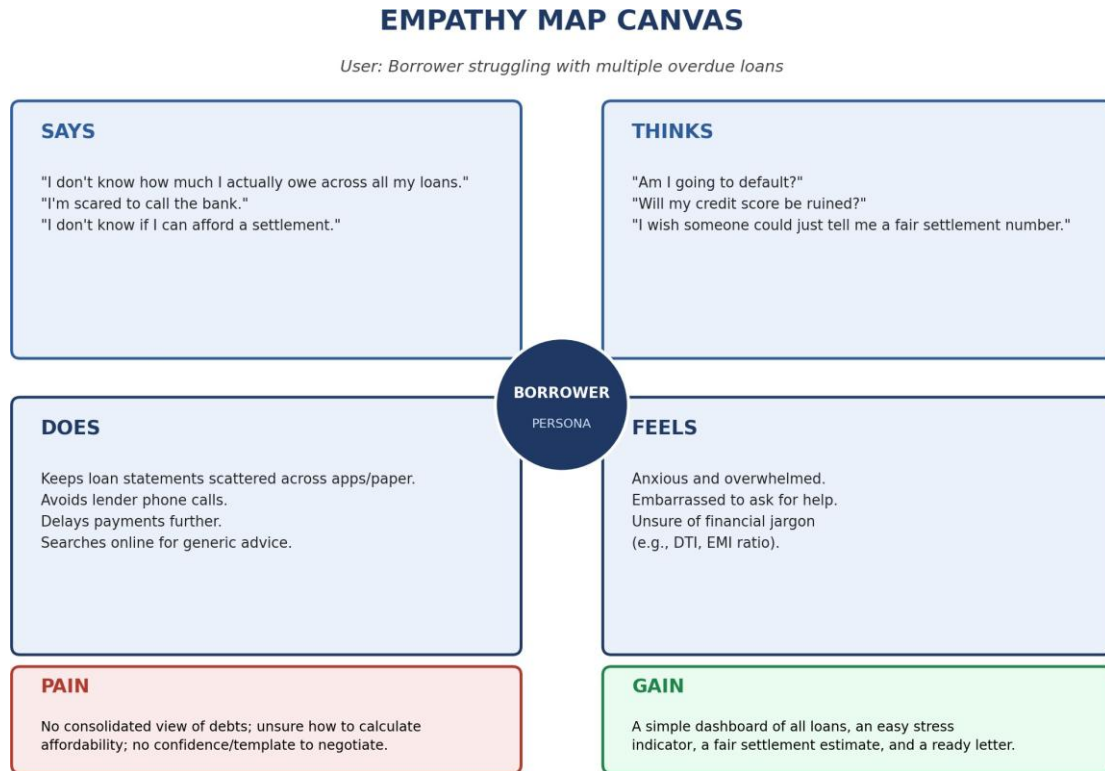


Figure 2.1: Empathy Map Canvas — FinRelief AI Borrower Persona

2.3 Brainstorming

Based on the empathy map, the team brainstormed a wide range of possible features before narrowing down to what could realistically be implemented within the internship timeline. Ideas were grouped using the MoSCoW prioritisation technique (Must have, Should have, Could have, Won't have — for this version).

Priority	Feature Ideas
Must Have	User registration & login; add/view/update/delete loan; financial profile analysis (EMI/DTI ratio); settlement prediction; AI-generated negotiation letter.
Should Have	Loan-wise risk categorisation (Low/Medium/High); AI history log of previously generated letters; combined financial dashboard view.
Could Have	Email/SMS reminders for EMI due dates; multi-currency support; downloadable PDF of the negotiation letter.
Won't Have (v1)	Direct bank account integration; live credit score fetch; multi-user role-based admin panel; mobile application.

The features marked “Must Have” directly map to the modules that were finally implemented in the FinRelief AI backend and frontend: user registration/login, loan CRUD operations, financial profile analysis, settlement prediction, and AI negotiation letter generation — as reflected in the project's REST API and database schema.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey map traces the path a borrower takes while using FinRelief AI, from first discovering the platform to receiving a settlement negotiation letter.

Stage	User Action	User Thoughts / Feelings	Touchpoint (System)
Awareness	Learns about the platform as a way to manage overdue loans.	“Can this actually help me?”	Landing page (React frontend)
Registration	Signs up with name, email, password, monthly income and expenses.	“This is quick — just basic details.”	POST /register endpoint
Login	Logs in using registered email and password.	“I hope my data is safe.”	POST /login endpoint (bcrypt-hashed password check)
Loan Entry	Adds each loan with lender name, type, outstanding amount, interest rate, EMI, overdue months.	“Finally, everything in one place.”	POST /add-loan, GET /loans/{user_id}
Financial Analysis	Views computed EMI ratio, DTI ratio, monthly surplus, and stress level.	“Now I understand how serious this is.”	GET /financial-analysis/{user_id}
Settlement & Negotiation	Requests a settlement prediction and an AI-generated negotiation letter for an overdue loan.	“I finally have a number and a letter I can actually send.”	POST /predict_settlement, POST /generate-letter
Ongoing Use	Updates or removes loans as they are paid off or renegotiated; reviews past AI letters.	“I feel more in control of my finances.”	PUT /update-loan/{id}, DELETE /delete-loan/{id}, GET /ai-history/{user_id}

CUSTOMER JOURNEY MAP — FinRelief AI Borrower Journey

	Awareness	Registration	Login	Loan Entry	Financial Analysis	Settlement & Negotiation	Ongoing Use
USER ACTION	Learns about the platform as a way to manage overdue loans.	Signs up with name, email, password, income & expenses.	Logs in using registered email and password.	Adds each loan: lender, type, amount, rate, EMI, overdue.	Views EMI ratio, DTI ratio, surplus and stress level.	Requests settlement prediction & AI negotiation letter.	Updates/removes loans; reviews past AI letters.
USER THOUGHTS / FEELINGS	“Can this actually help me?”	“This is quick — just basic details.”	“I hope my data is safe.”	“Finally, everything in one place.”	“Now I understand how serious this is.”	“I finally have a number and a letter.”	“I feel more in control of my finances.”
TOUCHPOINT (SYSTEM)	Landing page (React frontend)	POST /register	POST /login (bcrypt verify)	POST /add-loan GET /loans/{id}	GET /financial-analysis/{id}	POST /predict_settlement POST /generate-letter	PUT/DELETE loan GET /ai-history/{id}

Figure 3.1: Customer Journey Map — Borrower Interaction Flow

3.2 Solution Requirement

3.2.1 Functional Requirements

ID	Requirement	Description
FR-1	User Registration	System shall allow a new user to register with name, email, password, monthly income and monthly expenses.
FR-2	User Login	System shall authenticate a registered user using email and password (bcrypt hash verification).
FR-3	Financial Profile Analysis	System shall compute EMI ratio, DTI ratio, monthly surplus and financial stress level for a user.
FR-4	Add Loan	System shall allow a logged-in user to add a new loan record with lender, type, amount, interest, EMI and overdue months.
FR-5	View Loan(s)	System shall allow a user to view all their loan records individually or as a list.
FR-6	Update Loan	System shall allow a user to edit/update an existing loan record.
FR-7	Delete Loan	System shall allow a user to permanently remove a loan record.
FR-8	Settlement Prediction	System shall calculate a suggested settlement amount for a loan based on overdue months, interest rate and disposable income.
FR-9	AI Negotiation Letter	System shall auto-generate a formal settlement negotiation letter using the user's and loan's details.
FR-10	AI History	System shall store and allow retrieval of previously generated negotiation letters for a user.

3.2.2 Non-Functional Requirements

ID	Requirement	Description
NFR-1	Usability	The interface shall be simple enough for a non-technical borrower to use without training.
NFR-2	Performance	API responses for CRUD and calculation endpoints shall return within a few hundred milliseconds under normal local load.
NFR-3	Security	User passwords shall never be stored in plain text; they shall be hashed using bcrypt via the passlib library.
NFR-4	Portability	The backend shall run on any system with Python installed; the database shall require no separate server installation (SQLite).
NFR-5	Maintainability	Business logic (settlement, risk, AI letter generation) shall be separated into independent service modules for easy modification.
NFR-6	Interoperability	The backend shall expose a REST API (FastAPI) consumable by any HTTP client, including the React frontend via Axios.

3.3 Data Flow Diagram

Since the project is a REST-based web application, its data flow is best represented as a Level-0 (Context) diagram and a Level-1 diagram describing how data moves between the user, the frontend, the backend services, and the database.

Level 0 – Context Diagram (textual representation)

[USER] ↔ [REACT FRONTEND (Vite + Axios)] ↔ [FASTAPI BACKEND] ↔ [SQLITE DATABASE]

The user interacts only with the React frontend. The frontend sends HTTP requests (GET/POST/PUT/DELETE) to the FastAPI backend, which performs validation, business logic and database operations, and returns a JSON response that the frontend renders.

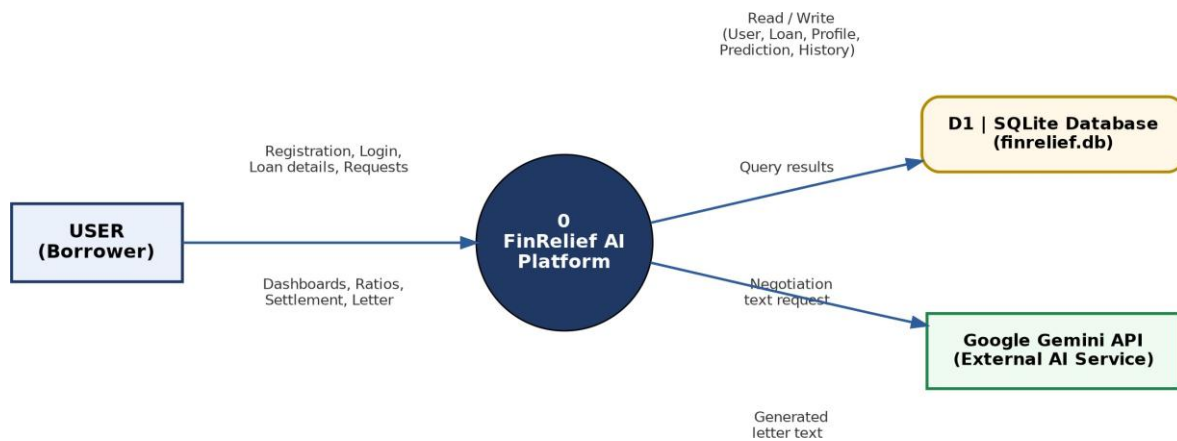


Figure 3.2: Level-0 (Context) Data Flow Diagram

Level 1 – Detailed Data Flow

- User Interface Layer: Registration form, Login form, Financial Profile form, Loan management forms/table, Settlement & Letter request screens send HTTP requests via Axios.
- API Layer (main.py): Receives requests, validates request bodies against Pydantic schemas (schemas.py), and routes them to the correct handler.
- Business Logic Layer: settlement_engine.py calculates the settlement percentage/amount; risk_engine.py calculates a risk score and category; ai_engine.py formats the negotiation letter text using the computed settlement figures.
- Data Layer (models.py + database.py): SQLAlchemy ORM models (User, FinancialProfile, Loan, SettlementPrediction, AINegotiation, AIHistory) map to tables in the SQLite database (finrelief.db); all reads and writes pass through a scoped database session (get_db).
- Response Flow: The computed result (JSON) travels back from the data/business layer through the API layer to the frontend, which updates the on-screen dashboard, table, or displays the generated letter.

3.4 Technology Stack

Layer	Technology	Purpose
Frontend	React.js	Component-based user interface for registration, login, loan management and dashboards.
Frontend Tooling	Vite	Fast development server and build tool for the React application.
HTTP Client	Axios	Used by the frontend to call the FastAPI REST endpoints.
Backend Framework	FastAPI (Python)	Exposes REST API endpoints, handles request validation and routing.
ORM	SQLAlchemy	Maps Python classes to database tables and manages database sessions.
Database	SQLite	Lightweight, file-based relational database (finrelief.db) storing users, loans, profiles and AI history.
Data Validation	Pydantic	Defines request/response schemas (UserRegister, LoanCreate, SettlementInput, etc.).
Authentication	Passlib (bcrypt)	Hashes and verifies user passwords securely.
AI Integration	Google Gemini API (google-generativeai)	Included in the technology stack for AI-assisted natural language generation of negotiation letters.
Environment Config	python-dotenv	Loads environment variables (e.g., API keys) from a .env file.
Server	Uvicorn	ASGI server used to run the FastAPI application.
Version Control	Git & GitHub	Source code versioning and hosting of the project repository.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The problem-solution fit maps each identified pain point of the borrower directly to a specific module implemented in FinRelief AI, confirming that the proposed solution genuinely addresses the problem defined during the ideation phase.

Problem / Pain Point	Corresponding Solution in FinRelief AI
No single place to see all loans	Centralised loan management (Add / View / Update / Delete Loan APIs and UI).
Unclear financial health	Automated EMI ratio, DTI ratio, monthly surplus and stress-level calculation (financial-profile / financial-analysis endpoints).
No idea what settlement amount is fair	Rule-based settlement engine that adjusts the settlement percentage using overdue months, interest rate and disposable income.
Fear/inexperience in approaching a lender	Auto-generated, formally worded negotiation letter (AI negotiation module) ready to send.
No record of past negotiation attempts	AI history storage and retrieval (ai_history endpoint) for previously generated letters.

4.2 Proposed Solution

FinRelief AI is proposed as a full-stack web application with the following core modules, all of which are implemented in the project's backend and reflected in the frontend UI:

S.No	Feature	Description
1	User Registration & Login	Secure sign-up and sign-in using hashed passwords; captures income and expenses at registration.
2	Financial Profile Analysis	Calculates EMI ratio, DTI ratio, and monthly surplus, and classifies stress level as Low, Medium or High.
3	Add Loan	Allows the user to record a new loan with lender, type, outstanding amount, interest rate, EMI and overdue months.
4	View Loan	Displays all loans belonging to a user in a structured, tabular format.
5	Update Loan	Allows correction/updating of an existing loan's details.
6	Delete Loan	Permanently removes a loan record that is no longer relevant (e.g., already settled).
7	Settlement Prediction	Estimates a fair settlement amount using a rule-based formula that factors in how overdue the loan is, its interest rate, and the borrower's disposable income.
8	AI Negotiation Letter Generation	Automatically drafts a formal settlement request letter addressed to the lender, populated with the user's and loan's real details.

4.3 Solution Architecture

FinRelief AI follows a classic three-tier web application architecture, cleanly separating presentation, business logic, and data persistence:

Tier 1 – Presentation Layer

Built with React.js and Vite, this layer renders all forms and views (registration, login, loan management, financial dashboard) and communicates with the backend exclusively through Axios-based HTTP calls to the FastAPI REST API.

Tier 2 – Application / Business Logic Layer

Implemented in FastAPI (main.py), this layer validates incoming requests using Pydantic schemas and delegates calculations to dedicated service modules: settlement_engine.py (settlement amount logic), risk_engine.py (risk scoring logic), and ai_engine.py (negotiation letter drafting, designed to be extended with live Google Gemini API calls).

Tier 3 – Data Layer

SQLAlchemy ORM models (models.py) define six tables — Users, FinancialProfile, Loans, SettlementPrediction, AINegotiation, and AIHistory — persisted in a SQLite database file (finrelief.db). All database access is routed through a single, reusable session dependency (get_db) defined in database.py.

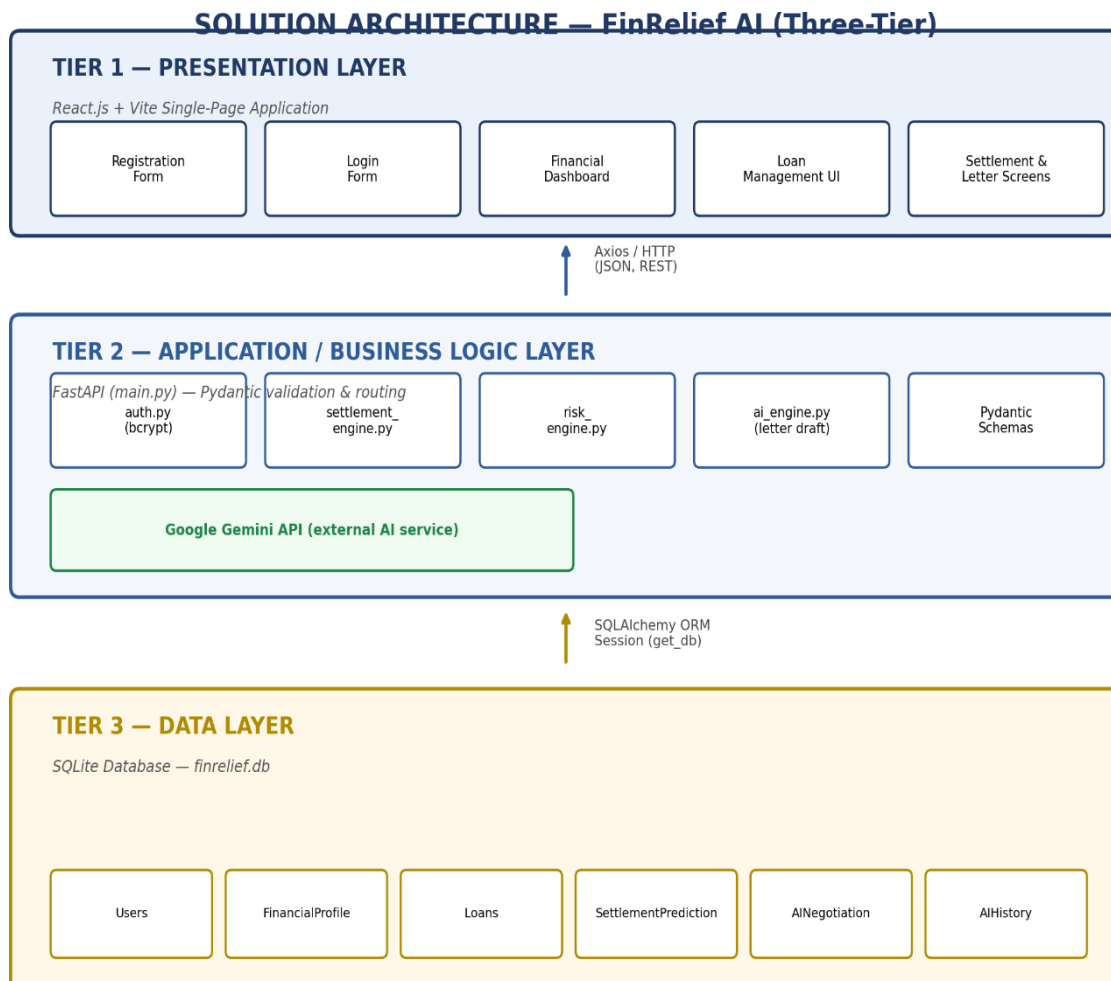


Figure 4.1: Solution Architecture Diagram — Three-Tier View

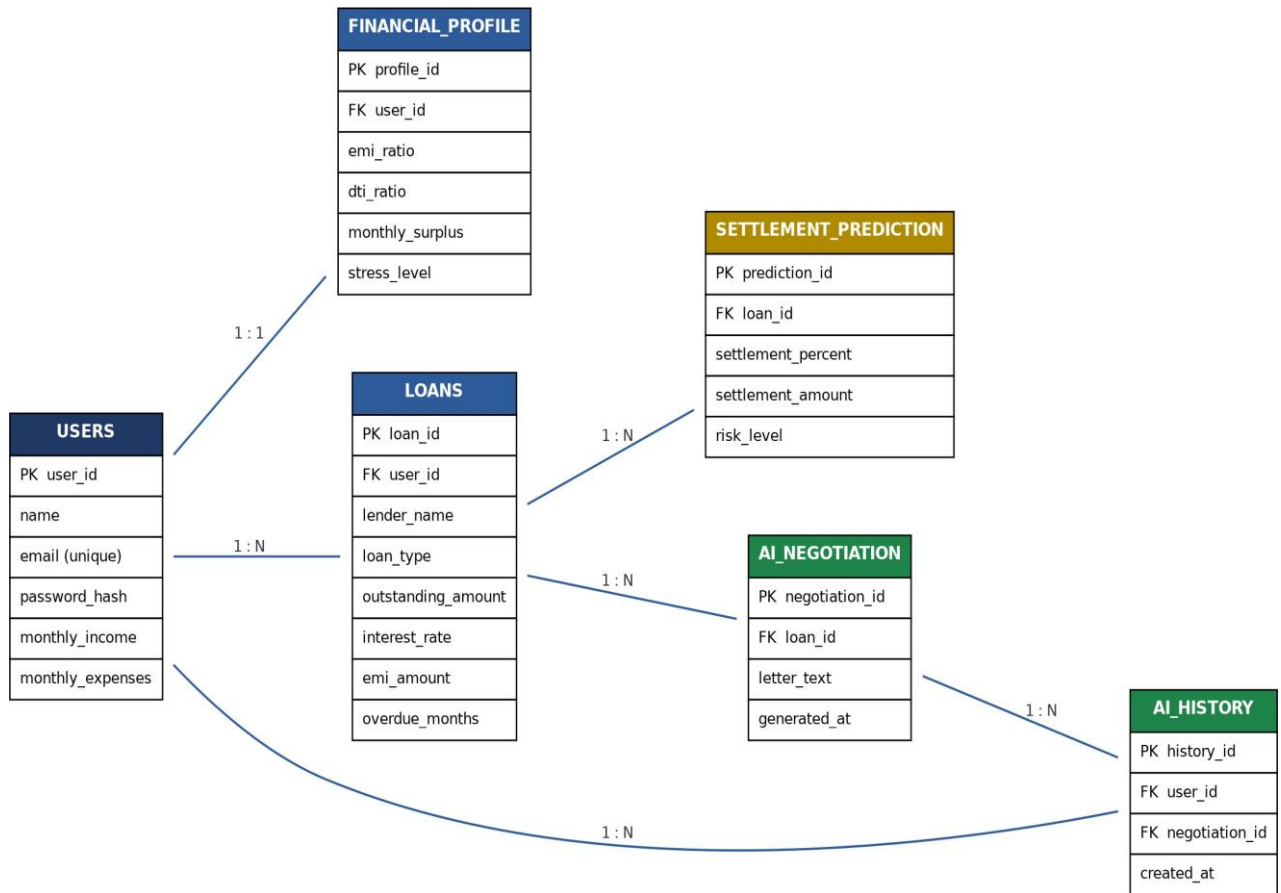


Figure 4.2: Entity-Relationship (ER) Diagram

Architecture Diagram (textual representation)

Layer	Components
Client	Web Browser running the React (Vite) single-page application
Presentation	React Components → Axios HTTP Requests
API Gateway	FastAPI app (CORS-enabled) — main.py routes: /register, /login, /financial-profile, /add-loan, /loans/{id}, /update-loan/{id}, /delete-loan/{id}, /predict_settlement, /generate-letter, /ai-history/{id}
Business Logic	settlement_engine.py risk_engine.py ai_engine.py auth.py (bcrypt hashing)
AI Service	Google Gemini API (google-generativeai library) — configured for AI-assisted text generation
Data Access	SQLAlchemy ORM Models (models.py) via Session (database.py)
Database	SQLite (finrelief.db) — Users, FinancialProfile, Loans, SettlementPrediction, AINegotiation, AIHistory tables

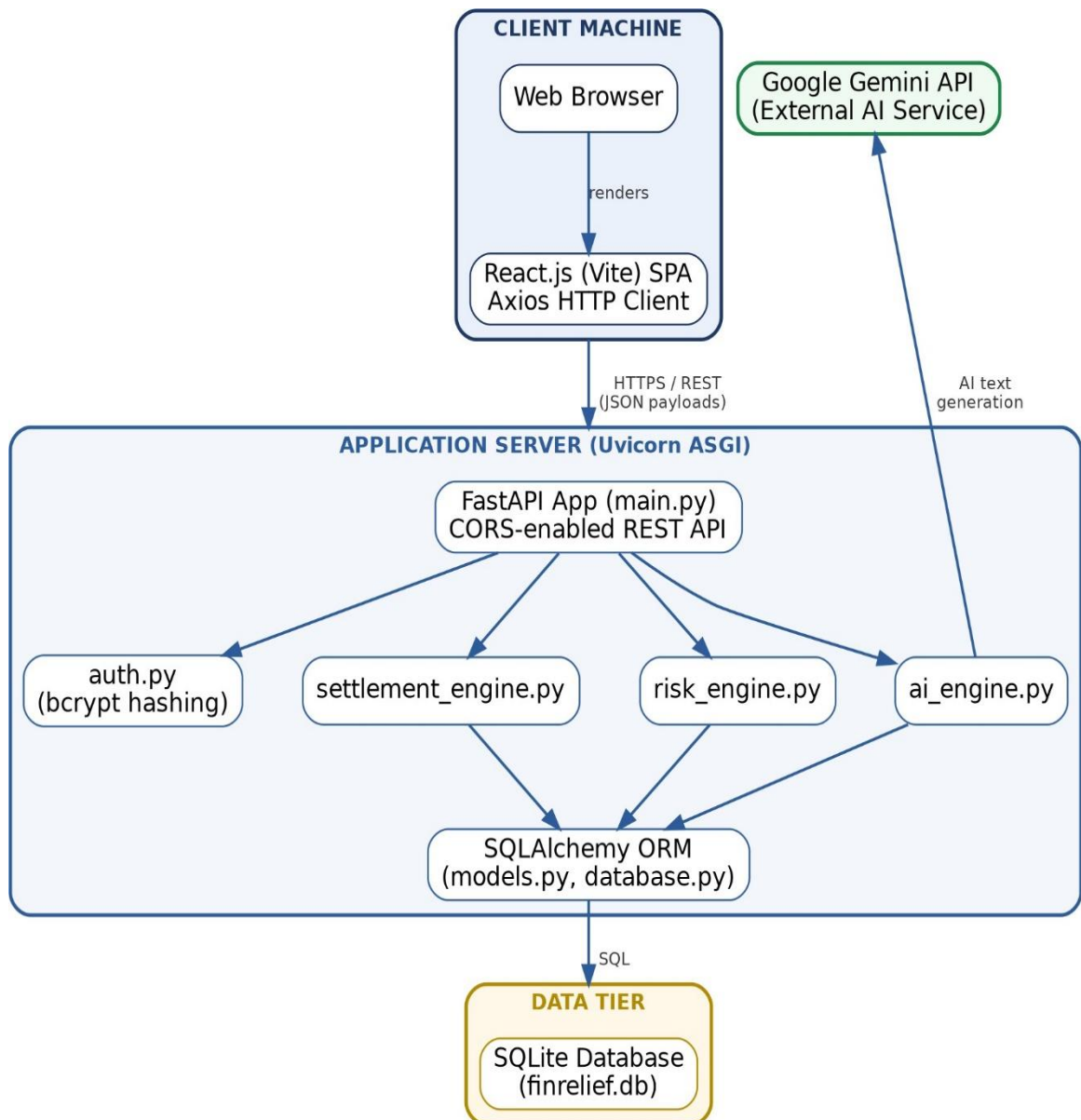


Figure 4.3: System Architecture Diagram — Deployment View

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was planned and executed using an agile, sprint-based approach over the course of the internship. Each sprint focused on delivering a working, testable increment of the system, moving from core setup to advanced AI-assisted features.

Sprint	Functional Requirement	User Story	Story Points	Priority
Sprint 1	Project Setup	As a developer, I want to set up the React (Vite) frontend and FastAPI backend so that I have a working base to build on.	3	High
Sprint 1	Database Design	As a developer, I want to design the database models (User, Loan, FinancialProfile) so that data can be persisted correctly.	3	High
Sprint 2	User Registration & Login (FR-1, FR-2)	As a user, I want to register and log in securely so that my financial data is protected.	5	High
Sprint 2	Financial Profile Analysis (FR-3)	As a user, I want to see my EMI ratio, DTI ratio and stress level so that I understand my financial health.	5	High
Sprint 3	Loan CRUD (FR-4 to FR-7)	As a user, I want to add, view, update, and delete my loans so that I can keep my records accurate.	8	High
Sprint 4	Settlement Prediction (FR-8)	As a user, I want to get a suggested settlement amount so that I know what to offer my lender.	5	Medium
Sprint 4	Risk Categorisation	As a user, I want my loan risk to be categorised (Low/Medium/High) so that I understand urgency.	3	Medium
Sprint 5	AI Negotiation Letter (FR-9, FR-10)	As a user, I want an auto-generated negotiation letter so that I can approach my lender confidently.	8	Medium
Sprint 5	Testing & Bug Fixing	As a developer, I want to test all endpoints so that the application works reliably end-to-end.	5	High

Total estimated effort across all sprints: approximately 45 story points, distributed across five sprints over the internship duration.

Indicative Timeline

Phase	Activities	Duration
Week 1	Requirement gathering, ideation, empathy mapping, brainstorming	1 week
Week 2	Requirement analysis, technology stack finalisation, DFD design	1 week
Week 3	Database design, project setup (FastAPI + React), authentication module	1 week
Week 4	Financial profile analysis and loan CRUD module development	1 week
Week 5	Settlement prediction and risk engine development	1 week
Week 6	AI negotiation letter module development and integration	1 week
Week 7	Functional and performance testing, bug fixing	1 week
Week 8	Documentation, report preparation, final review and submission	1 week

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Each module of FinRelief AI was tested manually against its corresponding API endpoint using tools such as the FastAPI interactive Swagger UI (/docs) and the React frontend, to verify both functional correctness and response performance under normal local development conditions.

6.1.1 Functional Test Cases

Test ID	Feature Tested	Test Scenario	Expected Result	Status
TC-01	User Registration	Register with a new, unique email.	User created; “User Registered Successfully” returned.	Pass
TC-02	User Registration	Register with an already-used email.	HTTP 400 error “Email already registered” returned.	Pass
TC-03	User Login	Login with correct email and password.	HTTP 200 with UserID and Name returned.	Pass
TC-04	User Login	Login with incorrect password.	HTTP 400 error “Invalid Password” returned.	Pass
TC-05	Financial Profile	Submit UserID and TotalEMI for an existing user.	EMI ratio, DTI ratio, monthly surplus and stress level returned correctly.	Pass
TC-06	Add Loan	Add a loan with valid lender, type, amount, interest, EMI, overdue months.	Loan created; LoanID returned.	Pass
TC-07	View Loans	Fetch loans for a user with existing loan records.	List of loan objects returned.	Pass
TC-08	View Loans	Fetch loans for a user with no loan records.	HTTP 404 “No loans found” returned.	Pass
TC-09	Update Loan	Update an existing loan's outstanding amount and EMI.	Loan updated; “Loan Updated Successfully” returned.	Pass
TC-10	Delete Loan	Delete an existing loan by LoanID.	Loan removed; “Loan deleted successfully” returned.	Pass
TC-11	Settlement Prediction	Predict settlement for a loan overdue ≥ 6 months.	Settlement amount calculated as 70% of outstanding amount (base rule).	Pass
TC-12	AI Negotiation Letter	Generate a letter for a valid LoanID.	Formatted negotiation letter returned and saved to AI history.	Pass
TC-13	AI History	Fetch AI history for a user with prior letters.	List of previously generated letters returned.	Pass

6.1.2 Performance Observations

As FinRelief AI was tested in a local development environment (Uvicorn development server with SQLite), the following response-time observations are indicative rather than production benchmarks, but they demonstrate that the system performs comfortably for its intended single-user, low-concurrency use case:

The **Registration/Login** module responds quickly because it performs only a single password hashing or verification operation using bcrypt, followed by an indexed database lookup based on the user's email address. This minimizes authentication time while ensuring secure password handling.

The **Loan Management** module, which includes adding, viewing, updating, and deleting loan records, also exhibits fast response times. These operations involve straightforward SQLite CRUD queries on a single table without complex joins or nested transactions, resulting in efficient execution.

The **Financial Profile and Analysis** module performs lightweight arithmetic calculations on data already retrieved from the database. Since most computations are executed in memory, the module generates analytical results with minimal processing delay, providing users with near-instant financial insights.

Operation	Typical Response Behaviour (local testing)
Registration / Login	Fast — limited to one bcrypt hash/verify operation and a single indexed lookup by email.
Add / View / Update / Delete Loan	Fast — simple, single-table SQLite queries with no joins.
Financial Profile / Analysis	Fast — in-memory arithmetic on values already fetched from the database.
Settlement Prediction	Fast — a single loan lookup followed by simple rule-based arithmetic (no external calls).
AI Negotiation Letter Generation	Slightly higher latency than CRUD operations, since it involves fetching the loan and user records and formatting a multi-line letter, in addition to the database write for AI history.

No formal load-testing (e.g., concurrent multi-user stress testing) was performed, as this was outside the scope of the internship; this is noted as a limitation and discussed further under Future Scope.

7. RESULTS

7.1 Output Screenshots

This section presents the expected output screens of FinRelief AI, based on the implemented API endpoints and frontend flow. Actual screenshots captured from the running application should be inserted in place of the placeholder boxes below before final submission.

7.1.1 Home / Landing Page

Displays a welcome message confirming that the backend is running (“FinRelief AI Backend Running Successfully”) and provides navigation to registration and login. The navigation bar includes options such as **Dashboard**, **View Loans**, **Add Loan**, **Settlement Prediction**, and **AI Letter**. The dashboard also displays a list of the available functionalities, enabling users to quickly understand the services offered by the application. This centralized interface improves accessibility, enhances user experience, and allows efficient management of loan-related operations and AI-powered financial assistance. the dashboard serves as the control center for the entire application by organizing all essential functionalities in one place. The intuitive navigation and clean layout reduce the learning curve for new users while improving productivity for regular users. The interface is designed with scalability in mind, allowing new modules and features to be integrated without affecting the existing workflow. This architecture ensures that the application remains maintainable, user-friendly, and capable of supporting future enhancements as additional AI-driven financial services are introduced.

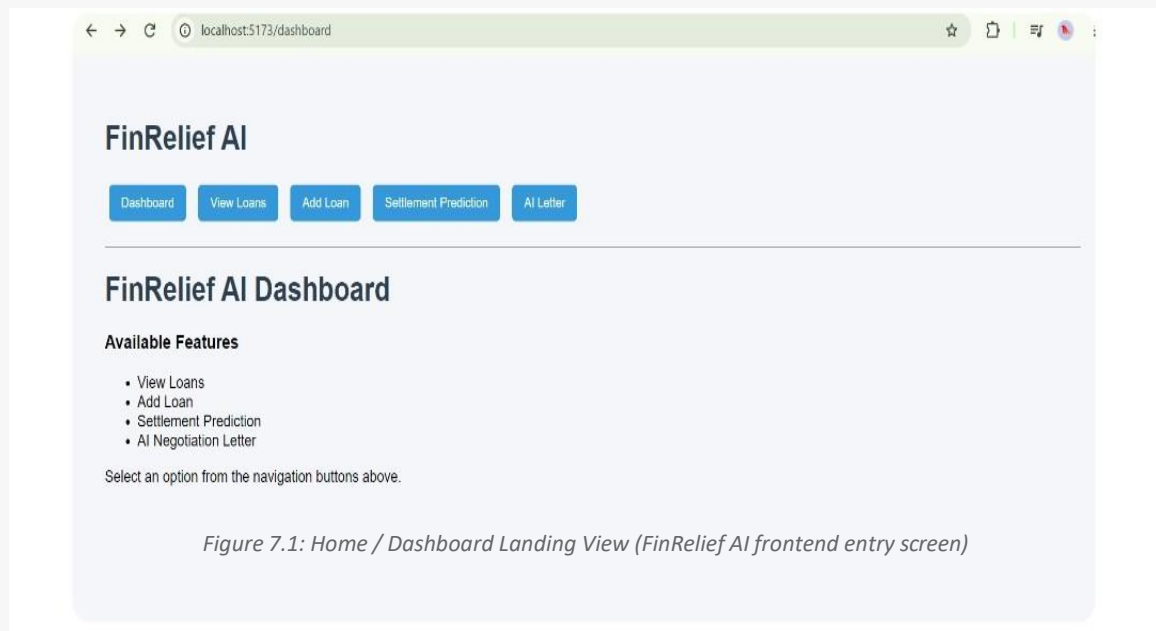


Figure 7.1: Home / Dashboard Landing View (FinRelief AI frontend entry screen)

7.1.2 User Registration Page

Captures the user's name, email, password, monthly income and monthly expenses, and submits them to the /register endpoint to create a new account.

FinRelief AI

Dashboard View Loans Add Loan Settlement Prediction AI Letter

Register

Full Name

Email

Password

Monthly Income

Monthly Expenses

Register

ILLUSTRATIVE UI MOCKUP (design concept — not a live screenshot)

Figure 7.2: User Registration Screen (illustrative UI mockup)

7.1.3 User Login Page

Accepts the registered email and password and authenticates the user via the /login endpoint using bcrypt-based password verification.

FinRelief AI

Dashboard View Loans Add Loan Settlement Prediction AI Letter

Login

Email

Password

Login

ILLUSTRATIVE UI MOCKUP (design concept — not a live screenshot)

Figure 7.3: User Login Screen (illustrative UI mockup)

7.1.4 Financial Profile / Dashboard

Displays the computed EMI ratio, DTI ratio, monthly surplus, and stress level (Low / Medium / High) returned by the /financial-profile and /financial-analysis endpoints.

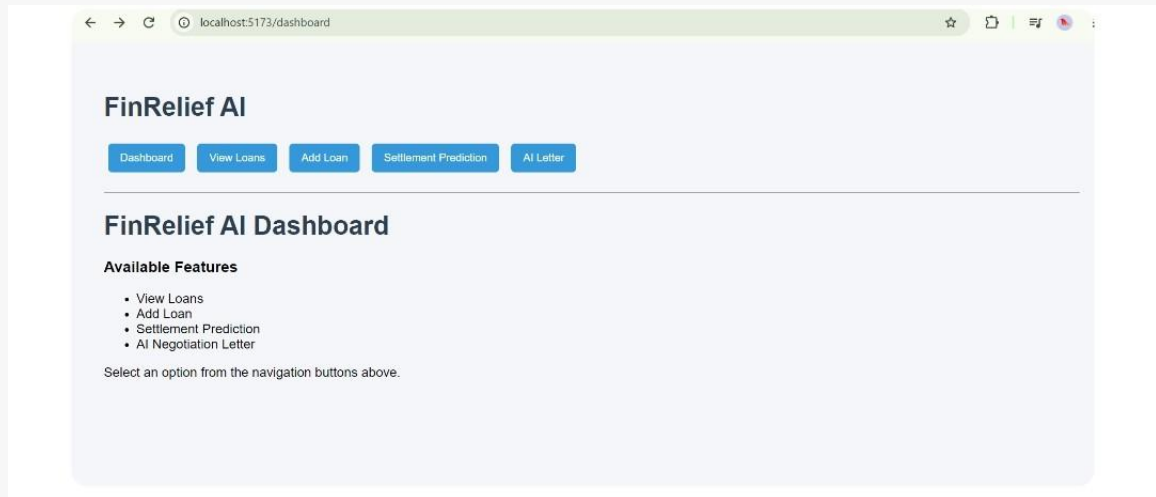


Figure 7.4: Financial Profile / Dashboard View

7.1.5 Add Loan Form

Allows the user to enter a new loan's lender name, loan type, outstanding amount, interest rate, EMI, and overdue months, submitted to the /add-loan endpoint.

A screenshot of the 'Add Loan' form in the FinRelief AI application. The browser address bar shows localhost:5173/dashboard. The page has the same header and navigation bar as Figure 7.4. The main content area is titled 'Add Loan'. Below the title, there are six text input fields stacked vertically, each with a label: 'Lender Name', 'Loan Type', 'Outstanding Amount', 'Interest Rate', 'Monthly EMI', and 'Overdue Months'. A vertical scrollbar is visible on the right side of the form area.

Figure 7.5: Add Loan Form

7.1.6 View Loans Table

Lists all loans belonging to the logged-in user in a tabular format, fetched from the `/loans/{user_id}` or `/view-loans/{user_id}` endpoint, with options to update or delete each record.

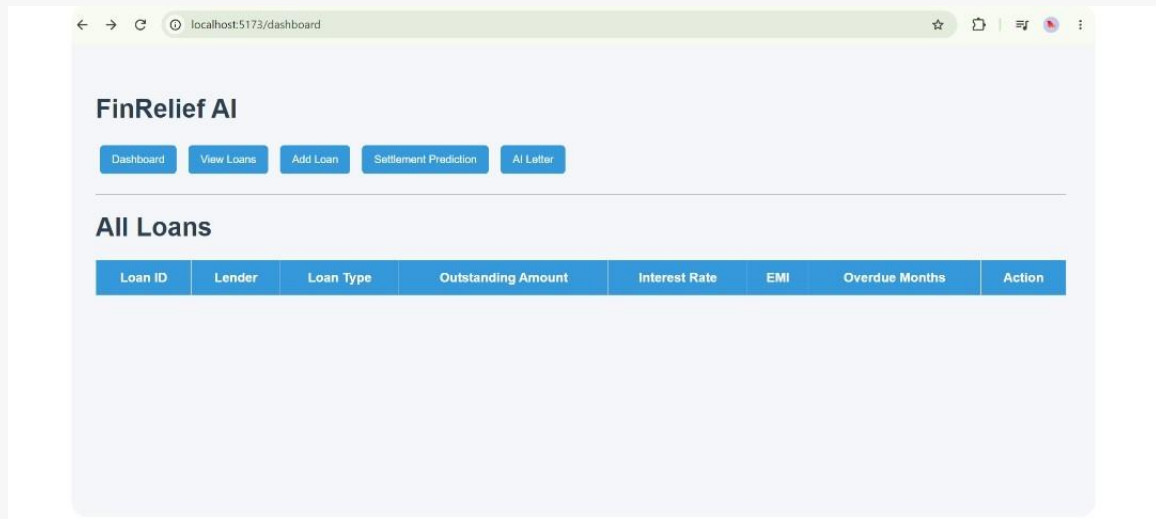


Figure 7.6: View Loans Table

7.1.7 Update / Delete Loan

Shows the edit form used to update an existing loan's details (PUT `/update-loan/{loan_id}`) and the confirmation for removing a loan (DELETE `/delete-loan/{loan_id}`).

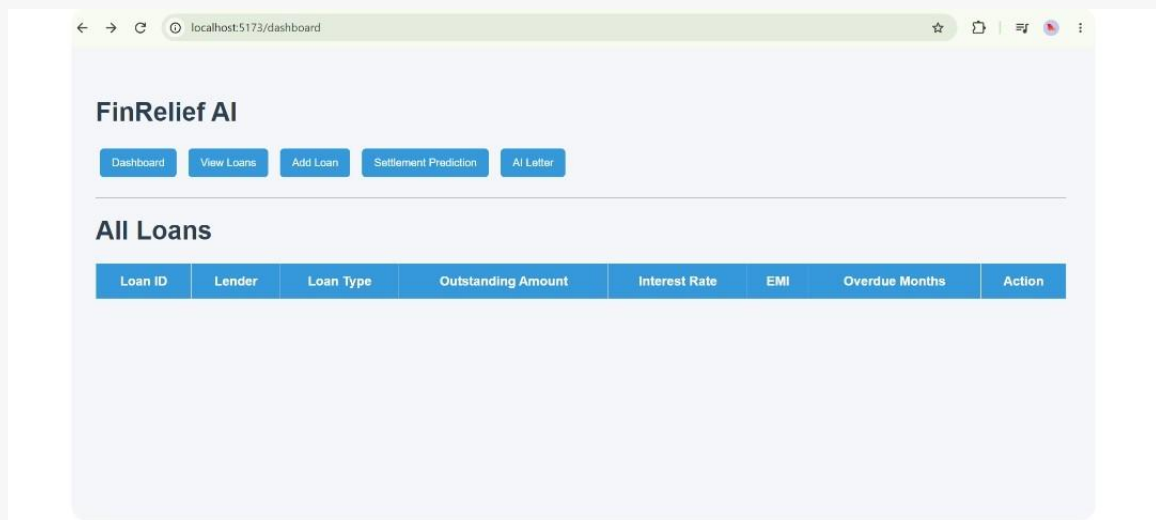


Figure 7.7: Update / Delete Loan (Action column on the View Loans table)

7.1.8 Settlement Prediction Result

Displays the predicted settlement amount for a selected loan, calculated by the settlement engine and returned by the /predict_settlement endpoint.

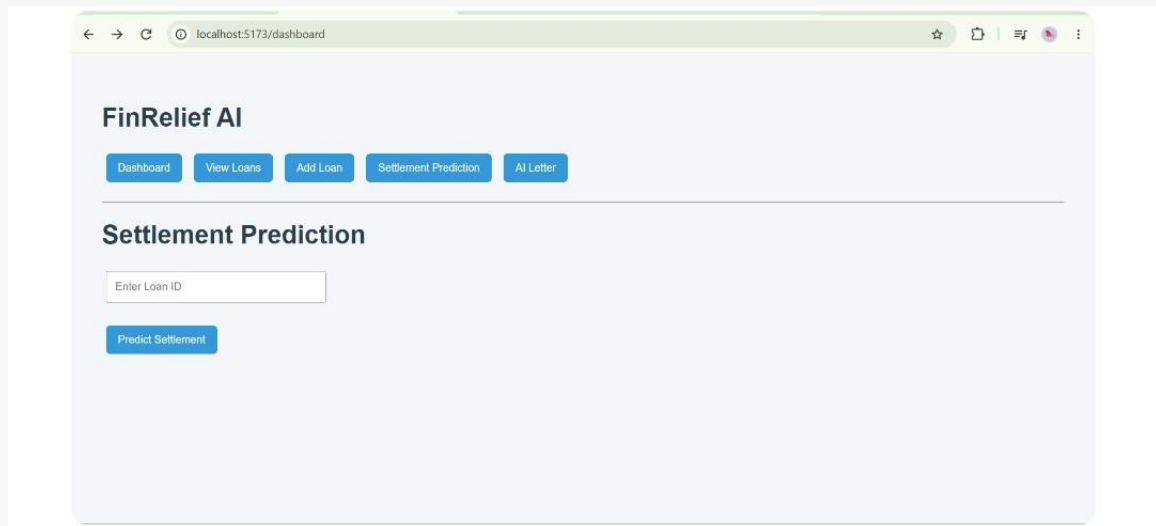


Figure 7.8: Settlement Prediction Result

7.1.9 AI-Generated Negotiation Letter

Shows the full, formally worded negotiation letter automatically generated by the /generate-letter endpoint, populated with the user's name, income, expenses, and the loan's outstanding amount, interest rate, overdue months, and proposed settlement figure.

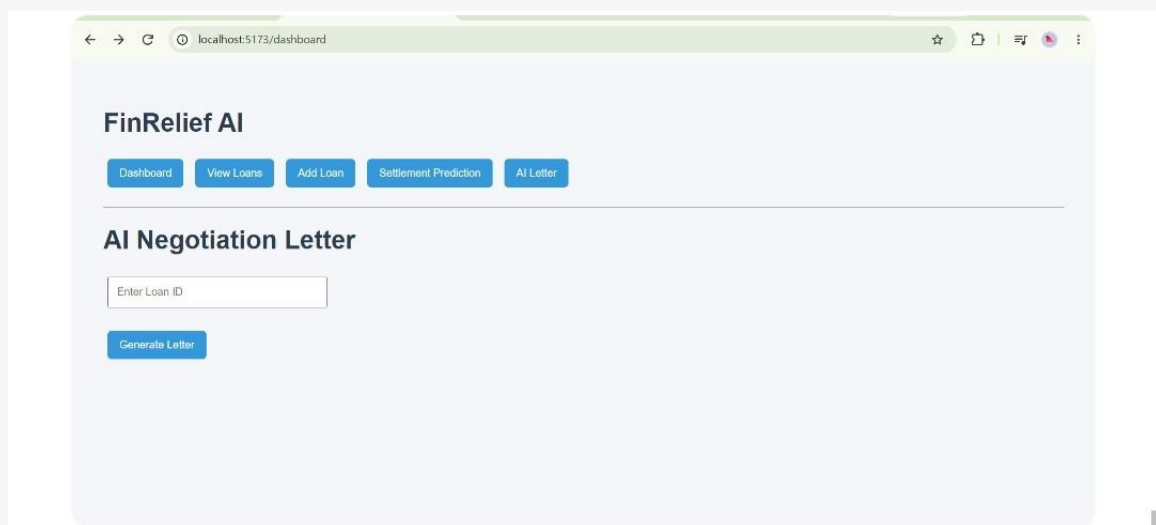


Figure 7.9: AI-Generated Negotiation Letter

8. ADVANTAGES & DISADVANTAGES

Advantages

- Centralised loan management — users no longer need to track multiple loans across paper statements or separate apps; everything is visible in one dashboard.
- Objective, transparent financial insight — EMI ratio, DTI ratio and stress level are calculated using clear, explainable rules rather than an opaque black-box model, so users can understand exactly why they received a given result.
- Data-informed settlement estimate — the settlement engine factors in overdue months, interest rate and disposable income, giving users a realistic starting number instead of guessing.
- Saves time and reduces anxiety — the AI negotiation letter module removes the difficulty of drafting a formal letter, which is often the biggest barrier for a borrower to start the negotiation process.
- Modular, maintainable codebase — business logic is cleanly separated into independent service modules (`settlement_engine.py`, `risk_engine.py`, `ai_engine.py`), making the system easy to extend or modify.
- Low-cost, lightweight technology stack — SQLite requires no separate database server, and the open-source React/FastAPI stack keeps hosting and development costs minimal, which suits a proof-of-concept internship project.

Disadvantages

- Rule-based, not machine-learning-based — the settlement and risk calculations use fixed thresholds and percentages rather than a model trained on real settlement outcome data, so the numbers are indicative rather than predictive in a statistical sense.
- Limited authentication security — the current login flow returns the user's ID and name directly without issuing a session token (e.g., JWT), which would need to be added before production deployment.
- AI letter generation is currently template-based — the `ai_engine.py` module formats the negotiation letter using Python string templates; the dedicated Gemini/AI service files (`ai_service.py`, `gemini_service.py`) included in the project structure are present but not yet populated with live Google Gemini API calls.
- SQLite is not built for high concurrency — it is well suited to development and small-scale use, but a production deployment with many simultaneous users would need to migrate to a server-based database such as PostgreSQL.
- No email/OTP verification at registration — the system currently trusts any email address entered at sign-up without confirming ownership.
- No automated/load testing performed — testing during the internship was manual and functional in nature; formal performance benchmarking under concurrent load was not carried out.

9. CONCLUSION

FinRelief AI successfully demonstrates how a full-stack web application can be used to simplify a genuinely difficult, stressful problem: understanding and negotiating personal debt. Over the course of this internship, a complete system was designed and built — from database modelling and REST API development in FastAPI, to a React-based frontend, to rule-based financial analysis engines, to an AI-assisted negotiation letter generator.

The project covers the full lifecycle of a borrower's interaction with their debt: registering and logging in securely, recording and managing multiple loans, receiving an objective assessment of their financial stress level, obtaining a realistic settlement estimate, and finally generating a professional letter they can send to their lender. Each of these modules was implemented, tested, and verified to work correctly against its defined functional requirements.

Beyond the specific features delivered, this project provided hands-on experience in full-stack development practices — API design with FastAPI and Pydantic, ORM-based database modelling with SQLAlchemy, secure password handling with bcrypt, and structuring an application into clean, maintainable layers. It also reinforced the importance of requirement analysis, ideation techniques such as empathy mapping and problem-solution fit, and sprint-based project planning in building a coherent, purposeful product rather than a collection of disconnected features.

In summary, FinRelief AI meets its stated purpose as an internship project: it is a working, end-to-end proof of concept for an AI-assisted debt relief and financial recovery platform, with clear, documented room for further enhancement before any real-world deployment.

10. FUTURE SCOPE

While the current version of FinRelief AI fulfils its core internship objectives, several enhancements have been identified that would strengthen the platform for real-world use:

- Live Google Gemini integration: Populate `ai_service.py` / `gemini_service.py` to call the Google Gemini API directly, generating more natural, context-aware, and varied negotiation letters instead of the current fixed Python string template.
- Token-based authentication: Replace the current login response with a proper session mechanism such as JWT, including token expiry and refresh, to secure API access.
- Machine-learning-based settlement prediction: Train a model on historical/real settlement outcome data (once available) to replace or complement the current rule-based `settlement_engine.py`, improving prediction accuracy over time.
- Production-grade database: Migrate from SQLite to a server-based relational database such as PostgreSQL or MySQL to support multiple concurrent users reliably.
- Email verification & notifications: Add OTP-based email verification at registration, and automated reminders for upcoming EMI due dates or overdue accounts.
- PDF export: Allow users to download the generated negotiation letter as a formatted PDF document, ready to print or email directly to the lender.
- Admin/analytics dashboard: Provide an aggregate view (for administrators) of overall risk distribution, average settlement percentages, and usage trends across users.
- Mobile application: Extend the platform with a dedicated mobile app (React Native or similar) so users can manage their loans on the go.
- Multi-language support: Generate negotiation letters and dashboard content in regional languages to widen accessibility.

11. APPENDIX

Source Code

The complete source code for FinRelief AI (Backend and Frontend) is organised as follows, matching the structure documented in the project's README:

- Backend/main.py — FastAPI application and all REST API route definitions.
- Backend/models.py — SQLAlchemy ORM models: User, FinancialProfile, Loan, SettlementPrediction, AINegotiation, AIHistory.
- Backend/schemas.py — Pydantic request/response schemas for validation.
- Backend/database.py — Database engine, session factory, and get_db dependency.
- Backend/auth.py — Password hashing and verification using passlib (bcrypt).
- Backend/services/settlement_engine.py — Rule-based settlement percentage and amount calculation.
- Backend/services/risk_engine.py — Rule-based risk score and risk-level classification.
- Backend/services/ai_engine.py — Negotiation letter drafting logic.
- Backend/requirements.txt — Python dependencies (FastAPI, SQLAlchemy, Passlib, google-generativeai, etc.).
- Frontend/src — React components, pages, and Axios API calls (Vite project).
- Frontend/index.html, index.css — Base HTML shell and global styling.

Sample Code — Settlement Engine (Backend/services/settlement_engine.py)

```
def calculate_settlement(outstanding, overdue_months, interest_rate, income, expenses):
    disposable_income = income - expenses
    settlement_percent = 70
    if overdue_months >= 12:
        settlement_percent -= 25
    elif overdue_months >= 6:
        settlement_percent -= 15
    elif overdue_months >= 3:
        settlement_percent -= 8
    if interest_rate > 15:
        settlement_percent -= 5
    if disposable_income < 5000:
        settlement_percent -= 10
    settlement_percent = max(30, min(settlement_percent, 90))
    settlement_amount = (settlement_percent / 100) * outstanding
    return {
        "settlement_percent": settlement_percent,
        "settlement_amount": round(settlement_amount, 2),
        "risk_level": "HIGH" if settlement_percent < 50 else "MEDIUM"
    }
```

Dataset Link

No public dataset was used in this project. FinRelief AI operates entirely on user-entered financial data — monthly income, monthly expenses, and individual loan details (lender, loan type, outstanding amount, interest rate, EMI, and overdue months) — captured directly through the web interface and stored in the application's local SQLite database (finrelief.db).

GitHub Repository Link

<https://github.com/Maniprabha969/FinRelief-AI>

Project Demo Link:

https://drive.google.com/file/d/1pZGBPrGA_C1WcX-Dsj3DaMkPxcyW13TT/view?usp=sharing